

Git & GitHub

Tutoriel essentiel — Projet ServiLoc · 6 membres · Commandes du quotidien

Projet	ServiLoc — Frontend React + Vite
Équipe	6 membres (1 référent, 1 intermédiaire, 4 débutants)
Contenu	Configuration, workflow quotidien, branches, PR, conflits, urgences
Prérequis	Git installé (git --version), compte GitHub, accès au repo
Document	31 mai 2026 — à lire entièrement avant de toucher au code

SOMMAIRE

1.	Configuration initiale	Git + identité + SSH + cloner le repo
2.	Les concepts fondamentaux	Repo, commit, branche, remote — ce que c'est vraiment
3.	Workflow quotidien	La routine de chaque jour en 6 étapes
4.	Branches	Créer, naviguer, supprimer — conventions ServiLoc
5.	Commits	Bonnes pratiques, messages, amender
6.	Synchronisation avec GitHub	push, pull, fetch — ne jamais perdre son travail
7.	Pull Requests	Créer, reviewer, merger — process ServiLoc
8.	Résoudre les conflits	Quand deux membres touchent le même fichier
9.	Commandes d'urgence	Annuler, récupérer, corriger les erreurs courantes
10.	Référence rapide	Toutes les commandes du projet sur une page

1. CONFIGURATION INITIALE

À faire une seule fois sur chaque ordinateur.

1.1 — Installer Git

Windows : télécharger Git Bash sur git-scm.com (cocher 'Git Bash Here' à l'installation). Mac : ouvrir Terminal et taper git — macOS proposera de l'installer automatiquement. Linux : `sudo apt install git`

1.2 — Configurer son identité

Ces informations apparaissent dans chaque commit. Utiliser le même email que GitHub.

```
# Remplacer par vos vraies infos

git config --global user.name "Madeleine Kamdem"

git config --global user.email "madeleine@email.cm"

# Vérifier que c'est bien enregistré

git config --global --list
```

1.3 — Cloner le repo ServiLoc

Cloner = télécharger le projet sur votre machine et le connecter au repo GitHub.

```
# Aller dans le dossier où vous voulez le projet (ex: Bureau)

cd ~/Desktop

# Cloner le repo (l'URL vous sera donnée par le référent M1)

git clone https://github.com/serviloc/serviloc-frontend.git

# Entrer dans le dossier

cd serviloc-frontend

# Vérifier que tout est là

git status

git log --oneline -5
```

- git clone crée automatiquement une connexion vers GitHub appelée 'origin'. Vous n'avez pas à le faire manuellement.

1.4 — Configurer la branche par défaut

```
# S'assurer qu'on est sur develop (jamais travailler sur main directement)

git checkout develop

# Vérifier les branches disponibles

git branch -a
```

■ Si 'develop' n'existe pas encore, le référent M1 doit la créer en premier.

2. LES CONCEPTS FONDAMENTAUX

Avant de taper des commandes, comprendre ce que Git manipule réellement.

Repository (repo)	Le dossier complet du projet + tout son historique. Votre copie locale ET la copie sur GitHub sont deux repos synchronisés. Quand vous clonez, vous obtenez une copie complète.
Commit	Une photo du code à un instant T. Chaque commit a un identifiant unique (ex: a3f9c12), un message, un auteur, et une date. L'historique Git = une série de commits.
Branche	Une ligne de travail indépendante. Travailler sur une branche n'affecte pas les autres. C'est ce qui permet à 6 personnes de coder en parallèle sans se bloquer.
main / develop	main = code stable livré. develop = intégration du travail de l'équipe. Vous ne travaillez JAMAIS directement sur main ou develop — toujours sur votre branche feature.
remote (origin)	Le nom donné au repo GitHub. 'origin' = le repo ServiLoc sur GitHub. git push origin = envoyer vers GitHub. git pull origin = recevoir depuis GitHub.
Staging area	Zone de préparation avant un commit. git add = mettre un fichier en staging. git commit = photographier tout ce qui est en staging.
merge	Fusionner deux branches. Quand votre PR est approuvée, votre branche est mergée dans develop.
conflict	Quand deux personnes modifient la même ligne du même fichier. Git ne sait pas quelle version garder — il vous demande de décider. Section 8 de ce tutoriel.

3. WORKFLOW QUOTIDIEN

Ces 6 étapes constituent la routine de chaque session de travail. Les respecter à chaque fois évite 90% des problèmes.

1 Récupérer les dernières modifications

Toujours commencer par récupérer ce que les autres ont pushé. Ne jamais coder sur un code qui n'est pas à jour.

```
# S'assurer d'être sur sa branche feature
git checkout feature/dashboard-client-m6

# Récupérer les modifs de develop et les fusionner dans votre branche
git pull origin develop
```

2 Vérifier l'état de ses fichiers

Voir quels fichiers ont été modifiés, ajoutés ou supprimés.

```
git status

# Voir les modifications ligne par ligne
git diff

# Voir les modifs d'un fichier spécifique
git diff src/pages/client/Dashboard.jsx
```

3 Ajouter les fichiers modifiés en staging

Sélectionner ce qu'on veut inclure dans le prochain commit.

```
# Ajouter un fichier spécifique (recommandé)
git add src/pages/client/Dashboard.jsx

# Ajouter plusieurs fichiers
git add src/pages/client/Dashboard.jsx src/pages/client/NewDemand.jsx

# Ajouter TOUS les fichiers modifiés (à utiliser avec prudence)
git add .

# Vérifier ce qui est en staging
git status
```

4 Créer le commit

Photographier le code avec un message clair. Le message est lu par toute l'équipe.

```
# Format obligatoire : type(scope): description courte
git commit -m "feat(dashboard): add client stats cards with mock data"

# Types autorisés :
# feat    = nouvelle fonctionnalité
# fix     = correction de bug
# style   = changement visuel (CSS, Tailwind)
# refactor= réécriture sans changer le comportement
# chore   = tâche technique (config, dépendances)
```

5 Envoyer sur GitHub

Pousser sa branche sur GitHub pour la sauvegarder et la rendre visible à l'équipe.

```
# Premier push d'une nouvelle branche

git push -u origin feature/dashboard-client-m6

# Push suivants (plus court)

git push
```

6 Ouvrir une Pull Request (quand le travail est fini)

Aller sur GitHub → onglet Pull Requests → New Pull Request. Base: develop / Compare: votre branche. Section 7 pour le détail.

```
# Vérifier que tout est pushé avant d'ouvrir la PR

git status

# Doit afficher : nothing to commit, working tree clean
```

4. BRANCHES

4.1 — Convention de nommage ServiLoc

Toutes les branches suivent ce format : feature/[écran-ou-tâche]-[membre]

Nom de branche	Description
feature/composants-core-m1	Référent — composants de base S1
feature/auth-inscription-m2	Membre 2 — page inscription
feature/auth-otp-m2	Membre 2 — page OTP (branche séparée)
feature/nouvelle-demande-m3	Membre 3 — formulaire nouvelle demande
feature/dashboard-presta-m4	Membre 4 — dashboard prestataire
feature/connexion-m5	Membre 5 — pages connexion
feature/dashboard-client-m6	Membre 6 — dashboard client
fix/otp-timer-bug-m2	Correction de bug (prefix fix/)
chore/setup-axios-m4	Tâche technique (prefix chore/)

4.2 — Commandes de gestion des branches

Commande	Ce que ça fait
git branch	Lister les branches locales (étoile = branche actuelle)
git branch -a	Lister toutes les branches (locales + distantes)
git checkout -b feature/mon-ecran-m3	Créer une nouvelle branche ET y aller
git checkout feature/mon-ecran-m3	Aller sur une branche existante
git checkout develop	Retourner sur develop

Un commit = une seule chose logique. Pas un commit par fichier, pas un commit pour tout le travail de la journée. La bonne granularité : quand vous pouvez décrire le commit en une phrase courte.

- Bonne règle : si votre message de commit contient le mot 'et', c'est probablement deux commits. 'feat(auth): add form validation AND fix redirect' → deux commits séparés.

5.3 — Commandes de commit

Commande	Ce que ça fait
<code>git add src/pages/client/Dashboard.jsx</code>	Mettre un fichier en staging
<code>git add src/pages/</code>	Mettre tout un dossier en staging
<code>git add .</code>	Mettre TOUS les fichiers modifiés (avec prudence)
<code>git add -p</code>	Sélectionner des parties d'un fichier interactivement
<code>git status</code>	Voir les fichiers staged / modifiés / non-trackés
<code>git diff --staged</code>	Voir exactement ce qui va être commité
<code>git commit -m "feat(scope): description"</code>	Créer le commit avec son message
<code>git commit --amend</code>	Modifier le dernier commit (AVANT push uniquement)
<code>git log --oneline</code>	Voir l'historique des commits en résumé
<code>git log --oneline --graph</code>	Voir l'historique avec visualisation des branches
<code>git show a3f9c12</code>	Voir le détail d'un commit spécifique

6. SYNCHRONISATION AVEC GITHUB

6.1 — push : envoyer vers GitHub

```
# Premier push d'une branche (crée la branche sur GitHub)
git push -u origin feature/dashboard-client-m6

# Le -u crée le lien (tracking) une seule fois

# Push suivants sur la même branche
git push

# Forcer un push (DANGEREUX – uniquement si vous savez ce que vous faites)
# Ne jamais utiliser sur develop ou main
git push --force-with-lease
```

6.2 — pull : recevoir depuis GitHub

```
# Récupérer les dernières modifs de develop dans votre branche

# À FAIRE au début de CHAQUE session de travail
```

```
git pull origin develop

# Récupérer les modifs de votre propre branche (si vous travaillez sur 2 machines)
git pull
```

6.3 — fetch : voir sans télécharger

```
# Vérifier ce qui a changé sur GitHub SANS modifier votre code

git fetch origin

# Voir les commits de develop sur GitHub sans les appliquer

git log HEAD..origin/develop --oneline
```

■ Règle d'or : toujours pull avant de push. Si vous poussez sans avoir récupéré les dernières modifs, Git refusera votre push et vous devrez pull d'abord.

6.4 — Vérifier l'état du repo

Commande	Ce que ça fait
<code>git remote -v</code>	Voir les connexions remote configurées (origin)
<code>git fetch --all</code>	Récupérer les infos de toutes les branches distantes
<code>git branch -vv</code>	Voir le statut de chaque branche vs GitHub
<code>git log origin/develop --oneline</code>	Voir l'historique de develop sur GitHub

7. PULL REQUESTS (PR)

Une Pull Request = demande de merge de votre branche dans develop. C'est sur GitHub, pas en ligne de commande. Le référent M1 valide toutes les PR.

7.1 — Créer une PR (étapes GitHub)

1	Vérifier que tout est pushé	<code>git status</code> → 'nothing to commit'. <code>git push</code> pour s'en assurer.
2	Aller sur GitHub → onglet Pull Requests	Cliquer sur 'New Pull Request'
3	Choisir les branches	base: develop ← compare: feature/votre-branche
4	Remplir la description	Titre : 'feat(auth): page inscription complète' Description : liste des écrans ajoutés, captures d'écran si possible, notes pour le reviewer
5	Assigner le reviewer	Toujours assigner le Référent M1 comme reviewer obligatoire.
6	Créer la PR	Cliquer 'Create Pull Request'. Prévenir M1 sur WhatsApp/Slack.

7.2 — Reviewer une PR (pour M1)

```
# Récupérer la branche du membre pour la tester localement
```

```
git fetch origin

git checkout feature/dashboard-client-m6

npm run dev    # Tester que ça tourne

# Revenir sur sa propre branche après review

git checkout feature/composants-core-m1
```

Sur GitHub : approuver ('Approve') ou demander des modifications ('Request changes'). Toujours laisser un commentaire même si tout est bon.

7.3 — Après merge de votre PR

```
# Récupérer develop mis à jour

git checkout develop

git pull origin develop

# Supprimer votre branche locale (elle est mergée, plus besoin)

git branch -d feature/dashboard-client-m6

# Créer une nouvelle branche pour la prochaine tâche

git checkout -b feature/notation-prestataire-m6
```

8. RÉSOUDRE LES CONFLITS

Un conflit apparaît quand deux membres modifient la même ligne du même fichier. Ce n'est pas une erreur — c'est Git qui vous demande de décider quelle version garder.

8.1 — Quand ça arrive

```
# Vous faites un pull et Git affiche :

CONFLICT (content): Merge conflict in src/components/Sidebar.jsx

Automatic merge failed; fix conflicts and then commit the result.

# git status vous montre les fichiers en conflit :

both modified:    src/components/Sidebar.jsx
```

8.2 — Ce que Git écrit dans le fichier

```
<<<<<< HEAD    (votre version)

    <Sidebar role='client' activeItem='dashboard' />

=====

    <Sidebar role='client' activeItem='demands' />

>>>>>> origin/develop    (version de l'autre membre)
```

- Tout ce qui est entre <<<<<< et ===== = votre version. Tout ce qui est entre ===== et >>>>>> = la version de l'autre. Vous devez choisir (ou combiner les deux) et supprimer ces marqueurs.

8.3 — Résoudre étape par étape

```
# 1. Ouvrir le fichier conflictuel dans VS Code

code src/components/Sidebar.jsx

# VS Code affiche des boutons : 'Accept Current' / 'Accept Incoming' / 'Accept Both'
# Choisir la bonne version (ou éditer manuellement)

# 2. Supprimer les marqueurs <<<, ==, >>> du fichier

# 3. Sauvegarder le fichier

# 4. Marquer le conflit comme résolu

git add src/components/Sidebar.jsx

# 5. Finaliser le merge

git commit -m "fix: resolve merge conflict in Sidebar component"
```

- Meilleure prévention : contacter M1 avant de toucher /src/components/. Les conflits arrivent quand deux personnes modifient le même fichier sans se coordonner. Le standup quotidien est là pour éviter ça.

9. COMMANDES D'URGENCE

Ces commandes permettent de corriger les erreurs les plus courantes. Lire attentivement AVANT d'exécuter — certaines sont irréversibles.

■ J'ai modifié un fichier par erreur et je veux annuler

```
# Annuler les modifications d'un fichier (revenir à la dernière version commitée)

git checkout -- src/pages/client/Dashboard.jsx

# Annuler TOUS les fichiers modifiés (non stagés)

git checkout -- .

# ATTENTION : irréversible si les fichiers n'ont pas été stagés
```

■ J'ai fait git add par erreur (enlever du staging)

```
# Enlever un fichier du staging (sans perdre les modifications)
git restore --staged src/pages/client/Dashboard.jsx

# Enlever tous les fichiers du staging
git restore --staged .
```

■ J'ai commité avec le mauvais message (avant push)

```
# Modifier le message du DERNIER commit (avant push uniquement)
git commit --amend -m "feat(dashboard): correct commit message"

# JAMAIS utiliser --amend après un push – ça crée des conflits pour l'équipe
```

■ J'ai commité des fichiers que je ne voulais pas

```
# Annuler le dernier commit en GARDANT les modifications dans les fichiers
git reset --soft HEAD~1

# Vos fichiers sont toujours là, juste dé-commités

# Annuler le dernier commit en SUPPRIMANT les modifications
git reset --hard HEAD~1

# ATTENTION : les modifications sont perdues définitivement
```

■ Je veux voir l'historique pour retrouver un commit

```
git log --oneline
git log --oneline --author='Madeleine'
git log --oneline --since='2026-05-28'

# Voir les changements d'un commit précis
git show a3f9c12
```

■ Je veux récupérer un fichier d'un commit précédent

```
# Récupérer un fichier tel qu'il était dans un commit précis
git checkout a3f9c12 -- src/pages/client/Dashboard.jsx

# Le fichier est maintenant en staging avec l'ancienne version

# Faire git commit pour sauvegarder
```

■ J'ai pushé sur la mauvaise branche

```
# Si vous avez pushé sur develop ou main par erreur → PRÉVENIR M1 IMMÉDIATEMENT  
  
# Ne pas essayer de corriger seul  
  
# Pour une branche normale : créer une PR de correction  
  
# ou demander à M1 de revenir en arrière via GitHub
```

■ Je veux sauvegarder mon travail en cours sans commit

```
# Mettre de côté les modifications en cours (stash)  
  
git stash  
  
# Récupérer les modifications mises de côté  
  
git stash pop  
  
# Utile pour changer de branche rapidement sans commiter
```

10. RÉFÉRENCE RAPIDE — TOUTES LES COMMANDES

Tout ce dont vous avez besoin au quotidien sur une seule page.

Configuration

Commande	Action
<code>git config --global user.name "Prénom Nom"</code>	Définir son nom
<code>git config --global user.email "email@cm"</code>	Définir son email
<code>git clone [url]</code>	Cloner le repo
<code>git remote -v</code>	Voir les remotes

État & historique

Commande	Action
<code>git status</code>	État des fichiers
<code>git diff</code>	Voir les mods non stagées
<code>git diff --staged</code>	Voir les mods stagées
<code>git log --oneline</code>	Historique des commits
<code>git log --oneline --graph</code>	Historique avec branches
<code>git show [hash]</code>	Détail d'un commit

Branches

Commande	Action
<code>git branch</code>	Lister les branches locales
<code>git branch -a</code>	Lister toutes les branches
<code>git checkout -b feature/mon-ecran-mx</code>	Créer et aller sur une branche
<code>git checkout [branche]</code>	Changer de branche
<code>git branch -d [branche]</code>	Supprimer une branche locale

Staging & commit

Commande	Action
<code>git add [fichier]</code>	Stager un fichier
<code>git add .</code>	Stager tous les fichiers
<code>git restore --staged [fichier]</code>	Déstager un fichier
<code>git commit -m "type(scope): description"</code>	Créer un commit
<code>git commit --amend</code>	Modifier le dernier commit

Synchronisation GitHub

Commande	Action
<code>git push -u origin [branche]</code>	Premier push d'une branche
<code>git push</code>	Push suivants
<code>git pull origin develop</code>	Récupérer develop dans sa branche

<code>git pull</code>	Récupérer sa propre branche
<code>git fetch origin</code>	Voir les changements sans les appliquer

Annuler / corriger

Commande	Action
<code>git checkout -- [fichier]</code>	Annuler les modifs d'un fichier
<code>git checkout -- .</code>	Annuler toutes les modifs
<code>git reset --soft HEAD~1</code>	Annuler dernier commit (garde les modifs)
<code>git reset --hard HEAD~1</code>	Annuler dernier commit (SUPPRIME les modifs)
<code>git stash</code>	Mettre de côté les modifs en cours
<code>git stash pop</code>	Récupérer les modifs mises de côté

RAPPEL — RÈGLES GIT SPÉCIFIQUES AU PROJET SERVILOC

Règle	Pourquoi
Ne jamais commiter sur main ou develop directement	Toujours travailler sur une branche feature/. main et develop sont protégées.
1 branche = 1 écran ou 1 tâche	Pas de mélange. Si vous travaillez sur 2 écrans → 2 branches.
PR obligatoire pour merger dans develop	Même si M1 vous dit 'c'est bon'. Le process existe pour une raison.
M1 valide toutes les PR	Créer la PR → prévenir M1 → attendre son approbation → merger.
Ne jamais force push sur develop ou main	<code>git push --force</code> sur une branche partagée écrase le travail des autres.
Commits en anglais	Convention internationale, compatible avec GitHub et les outils de review.